Gate 01 Reconciliation: Foundation & CI

Field	Value
Gate	01 — Foundation & CI
Spec version	Initial (created in commit f9bb90b)
Implementation date	2026-04-16
Reconciled by	Claude Code
Branch	gate/01-foundation

Summary

Gate 01 was implemented across two commits (`ecf65d1` initial implementation, `9901237` code quality fixes) and merged to main in `ad6f980` . All core deliverables were completed: Django project skeleton with split settings, Docker Compose stack, GitHub Actions CI, health endpoint, and pre-commit hooks. A few spec items were implemented with enhancements beyond what was specified, and one app (`coordination`) from the architecture proposal was omitted from `INSTALLED_APPS` -- this is addressed in Gate 02.

Acceptance Criteria Status

#	Criterion (from spec)	Status	Notes
1	`docker-compose up` starts cleanly with no errors	Implemented	Six services defined: `web`, `worker`, `beat`, `db`, `redis`, `minio`. Health checks on `db` and `redis`.
2	`pytest` passes against a PostGIS-enabled test database	Implemented	CI uses `postgis/postgis:16-3.4` service; `pyproject.toml` configures `DJANGO_SETTINGS_MODULE=config.settings.test` and `--reuse-db`.
3	GitHub Actions CI passes on a clean push	Implemented	`.github/workflows/ci.yml` with `lint` and `test` jobs.
4	`GET /api/v1/health/` returns 200	Implemented	Endpoint at `/api/v1/health/` returns 200 with `status`, `version`, `database`, `cache` fields.
5	Invoke @code-quality-reviewer on project structure and CI	Implemented	Commit `9901237` ("fix code quality review findings") shows this was done.

User Story Status

Story ID	Title	Status	Notes
BE-01-1	Health Check Endpoint	Implemented as specified	Returns 200 with `{"status":"ok","version":"0.1.0","database":"connected","cache":` Returns 503 with `"degraded"` on DB or cache failure. 4 tests cover no-auth, DB error, and cache error scenarios.

Deliverables Status

Deliverable	Status	Notes
Split settings (`base.py`, `dev.py`, `prod.py`)	Implemented with addition	`test.py` also created (not in spec but necessary).
Docker Compose stack (6 services)	Implemented	`web`, `worker`, `beat`, `db`, `redis`, `minio` all present with health checks.
GitHub Actions CI (ruff, mypy, pytest)	Implemented	Two jobs: `lint` (ruff check, ruff format, mypy) and `test` (pytest with PostGIS).
Apache-2.0 license file	Implemented	`LICENSE` present with Apache 2.0 text.
`.env.example` committed, `.env` in `.gitignore`	Implemented	Both present and correctly configured.
Health endpoint	Implemented	See BE-01-1 above.
pytest with PostGIS test DB	Implemented	CI uses `postgis/postgis:16-3.4`; test settings use `locmem` cache.
pre-commit hooks (ruff, mypy)	Implemented	`.pre-commit-config.yaml` with ruff (lint + format) and mypy (with typed stubs).

Deviations

1. Test settings file added (not in spec)

- **Spec said:** Split settings: `base.py`, `dev.py`, `prod.py`
- **Implementation does:** Also includes `test.py` with in-memory cache and fast password hashing
- **Reason:** Standard Django practice; needed for pytest configuration
- **Impact:** Positive -- faster tests, isolated test environment

2. Health endpoint returns additional fields

- **Spec said:** Response includes `status`, `version`, `database`, `cache`
- **Implementation does:** Returns exactly those four fields, plus uses `"degraded"` (not just `"error"`) for the top-level status on failure
- **Reason:** The spec's acceptance criteria say 503 with `database: "error"` -- the implementation does return `database: "error"` but the top-level `status` field uses `"degraded"` rather than repeating `"error"`. This is a sensible refinement.
- **Impact:** None -- the spec's acceptance criteria are satisfied

3. `coordination` app missing from INSTALLED_APPS

- **Spec said:** N/A (Gate 01 explicitly excludes domain logic)
- **Implementation does:** `INSTALLED_APPS` includes `accounts`, `species`, `populations`, `fieldwork`, `integration` but not `coordination`
- **Reason:** The architecture proposal lists 6 Django apps including `coordination`. However, Gate 01 spec explicitly says "No domain logic in this gate -- pure infrastructure." The app stubs were included for project structure but `coordination` was deferred.
- **Impact:** Must be created in a later gate. This is tracked as a known gap.

4. CI triggers on additional branch patterns

- **Spec said:** CI on "every push" and "PRs"
- **Implementation does:** Triggers on push to `main`, `gate/*`, `feat/*`, `fix/*`, `docs/*` branches; PRs to `main`
- **Reason:** Aligns with the project's git workflow conventions defined in CLAUDE.md
- **Impact:** Positive -- CI runs on all development branches

5. ruff format check added to CI

- **Spec said:** lint (`ruff`), type check (`mypy --strict`)
- **Implementation does:** CI runs `ruff check`, `ruff format --check`, and `mypy`
- **Reason:** Format checking is standard practice and was added alongside lint
- **Impact:** Positive -- enforces consistent formatting

6. PostGIS extension init SQL not explicitly present

- **Spec said:** "Enable PostGIS extension in the db service via init SQL (`CREATE EXTENSION IF NOT EXISTS postgis;`)"
- **Implementation does:** Uses the `postgis/postgis:16-3.4` Docker image which automatically enables PostGIS
- **Reason:** The official PostGIS Docker image creates the extension by default, making manual init SQL unnecessary
- **Impact:** None -- PostGIS is available; the approach is simpler

7. Additional dependencies beyond spec

- **Spec said:** Install `django`, `djangorestframework`, `psycopg2-binary`, `django-environ`, `gunicorn`, `celery`, `redis`, `django-storages`, `boto3`, `Pillow`
- **Implementation does:** Also includes `django-celery-beat`, `GDAL`, `djangorestframework-gis`, `django-mptt`, `pytest`, `pytest-django`, `ruff`, `mypy`, `django-stubs`, `djangorestframework-stubs`, `pre-commit`
- **Reason:** `django-celery-beat` needed for Celery Beat scheduler; `GDAL` / `djangorestframework-gis` needed for PostGIS; `django-mptt` for taxonomy hierarchy; dev/test tools for CI
- **Impact:** Positive -- all are required by the architecture proposal and CI configuration

Additions (not in spec)

Addition	Description	Justification
`config/celery.py`	Celery application configuration with autodiscover	Required for `worker` and `beat` Docker services to function

`config/wsgi.py`	WSGI application entry point	Required for Gunicorn to serve Django
`config/api_urls.py`	Separate URL router for API v1 namespace	Clean separation of admin and API URL routing
`accounts.urls` included in API	Auth URL namespace wired under `/api/v1/auth/`	Forward-looking inclusion for Gate 03
`accounts.backends.EmailBackend`	Custom auth backend referenced in settings	Forward-looking for email-based login (Gate 03)
Production security settings	`SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `HSTS` in `prod.py`	Security best practice; aligns with architecture proposal's TLS requirements
Password minimum length 12	`MinimumLengthValidator` with `min_length=12`	Matches architecture proposal's password policy
DRF configuration	Token + Session auth, pagination (50/page), JSON renderer	Standard DRF setup needed for health endpoint and future gates

Deferred Items

Item	Reason	Target Gate
`coordination` Django app	Not needed until conservation planning features	TBD (likely Gate 04+)
Frontend (Next.js)	Explicitly out of scope for Gate 01	Gate 07
Domain models	Explicitly out of scope	Gate 02
Auth beyond Django defaults	Explicitly out of scope	Gate 03

Technical Decisions Made During Implementation

1. **PostGIS Docker image instead of init SQL:** Used `postgis/postgis:16-3.4` which auto-enables extensions, rather than a vanilla PostgreSQL image with init scripts. Simpler and more maintainable.
2. **Separate `api_urls.py` router:** API endpoints are namespaced under `api/v1/` via a dedicated URL module, keeping the root `urls.py` clean. This supports API versioning.
3. **Database port mapping to 15432:** Docker Compose maps the PostgreSQL container's port 5432 to host port 15432 to avoid conflicts with local PostgreSQL installations.
4. **`django-environ` for settings:** Environment variables parsed via `django-environ` rather than raw `os.environ`, providing type casting and defaults.
5. **`--reuse-db` in pytest:** Configured to reuse the test database between runs for faster local development iteration.

6. **`BigAutoField` as default PK:** Set `DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField"` for future-proofing primary keys.

7. **Celery defaults to prod settings:** `celery.py` and `wsgi.py` default to `config.settings.prod`, overridden by `DJANGO_SETTINGS_MODULE` env var in Docker Compose.

Spec Updates Needed

1. **Add `test.py` to deliverables:** The split settings deliverable should list four files: `base.py`, `dev.py`, `prod.py`, `test.py`.

2. **Update PostGIS init approach:** Replace "Enable PostGIS extension via init SQL" with "Use official `postgis/postgis` Docker image" since the image handles extension creation.

3. **Add dev/test dependencies to core deps list:** The technical tasks list core dependencies but omits test/dev tooling (`pytest`, `pytest-django`, `ruff`, `mypy`, stubs, `pre-commit`) and GIS/model dependencies (`GDAL`, `djangorestframework-gis`, `django-mptt`, `django-celery-beat`).

4. **Clarify CI format checking:** The spec says "lint (ruff)" but the CI also runs `ruff format --check`. The spec should mention both linting and format enforcement.

5. **Note `coordination` app deferral:** The architecture proposal defines 6 apps; only 5 are stubbed in Gate 01. The spec or a future gate spec should explicitly plan for `coordination` app creation.